
KONZEPTPAPIER · AUTONOMER AGENTENBETRIEB

MqlKiScanner

Fünf kooperierende LLM-Rollen
übernehmen die Dauerbeobachtung
forensisch geprüfter MQL5-Signale:
tägliche Delta-Prüfung gegen
Algo-Profile, Marktdaten aus dem
MetaTrader — lückenlos protokolliert,
ohne Neubewertung.

tnickel · tnickel-ki.de · Stand: 22. September
2026

MQLKISCANNER · DOC 2026-AG-01

Inhaltsverzeichnis

1 • Zusammenfassung	2
2 • Ausgangslage: Der MqlKiScanner	2
2.1 • Ein forensischer Scanner für MQL5-Signale	2
2.2 • Die bestehende Datenbasis	3
2.3 • Der Restaufwand des Menschen	3
2.4 • Prinzipien als Fundament	4
3 • Die Idee: Beobachtung als Dienst	4
4 • Architektur des Agentenbetriebs	5
5 • Die fünf Rollen	6
6 • Das Signal-Dossier	7
7 • Der Tagesablauf: Trade-Delta-Prüfung	8
8 • Marktdaten aus dem MetaTrader	9
9 • Betriebsrhythmus und Kosten	10
10 • Governance: Leitplanken autonomer KI	11
11 • Datenmodell: sechs neue Tabellen	12
12 • Benutzeroberfläche	12
13 • Umsetzungsfahrplan	13
14 • Risiken und Gegenmaßnahmen	13
15 • Fazit und Ausblick	14

1 · Zusammenfassung

Der MqlKiScanner prüft Handelssignale der MQL5-Plattform forensisch: Er rekonstruiert Drawdowns aus den Roh-Trades, misst Exposure im Schockfall, weist Stop-Loss-Disziplin nach oder widerlegt sie und verdichtet alles zu einer Ampel-Bewertung. Bis heute wird jedes dieser Verfahren manuell gestartet, gelesen und nachverfolgt. Dieses Konzeptpapier beschreibt den nächsten Schritt: die Übergabe des operativen Tagesgeschäfts an einen **autonomen Agentenbetrieb** — fünf spezialisierte LLM-Rollen, die als Dienstprozess im selben Projekt zusammenarbeiten, den Markt beobachten, jedes Signal täglich auf neu hinzugekommene Trades prüfen und Auffälligkeiten melden.

Der Kerngedanke: Die ausführlichen KI-Analysen der Vergangenheit beschreiben je Signal, *wie der Handelsalgorithmus arbeitet* — Sessions, Positionsgrößen, Einstiegs- und Ausstiegslogik, Stop-Disziplin. Genau diese Beschreibungen werden zu maschinell prüfbareren **Algo-Profilen** verdichtet. Ein täglicher Abgleich der neuen Trades gegen das Profil beantwortet die eigentliche Überwachungsfrage: *Handelt das Signal noch wie es soll — oder hat sich heimlich etwas geändert?*

Zwei Grundregeln des Projekts bleiben unangetastet: **Der Code rechnet, das LLM interpretiert** — sämtliche Zahlen entstehen deterministisch in der Engine, Sprachmodelle zitieren sie nur. Und: **Die Engine bewertet** — Ampeln, Urteile und Scores ändert ausschließlich ein regulärer Scan-Lauf; Agenten beobachten und melden, bewerten aber niemals neu.



Der Fahrplan umfasst fünf Ausbaustufen (A bis E) von Fundament, Dossiers und Marktanbindung bis zum vollautonomen Monat. Der Bauplan ist im Projekt dokumentiert; dieses Papier erläutert Konzept, Architektur und Leitplanken in der Breite.

2 · Ausgangslage: Der MqlKiScanner

2.1 · Ein forensischer Scanner für MQL5-Signale

MQL5-Signale sind kopierbare Handelsstrategien Dritter. Die Plattform-Metrik „maximaler Drawdown“ genügt für eine Risikoentscheidung nicht: Sie kann niedriger ausfallen als der reale Intrade-Verlust, Martingale-Systeme verstecken sich hinter glatten Equity-Kurven, und Namen wie „Low Risk“ oder „Stable“ sind Marketing. Der MqlKiScanner prüft daher auf Rohdaten-Ebene — den exportierbaren Trade-Listen jedes Signals — und stellt die Frage des Projekts: **Ist der Schutz bewiesen oder nur behauptet?**

Vier Pflicht-Tests müssen bestehen, bevor ein Kandidat überhaupt positiv eingestuft werden kann:

Pflicht-Test	Was geprüft wird	Warnflag
Martingale-Signatur	Verhältnis $\text{Lot}(i+1)/\text{Lot}(i)$ nach Verlusten über die gesamte Historie	Median über 1,3-facher Erhöhung
Peak-Exposure	maximal gleichzeitig offene Positionen und aggregiertes Netto-Volumen, umgerechnet in USD-Risiko je Instrument (inklusive Schockszenario)	hohe Positionsanzahl, hohes Volumen, Schock über Kontogröße
Stop-Loss-Nachweis	Orderbuch-Beweis (SL/TP-Spalten, Trailing-Kommentare) oder statistisches Clustering der Verlustdistanzen	kein Nachweis — „nicht bewiesen“ ist eine Warnung, nie eine Entlastung
Drawdown-Nachbau	Nachbau der Kontokurve aus den Trades, Abgleich mit den Plattformwerten	Abweichung oder Verletzung der 30-Prozent-Schranke

Tabelle 1: Pflicht-Test-Batterie vor jeder positiven Einstufung

Das Ergebnis verdichtet sich zu einem Score (1 bis 10, niedrig ist gut) und einer Ampel über acht Einzelkriterien — darunter die harte Drawdown-Schranke von 30 Prozent, der Ertrag (Ziel über 5 % monatlich, stets nachrangig hinter dem Risiko) und der Stop-Nachweis. Eine kuratierte Ausschlussliste mit begründeten Fällen verhindert Wiederaufnahmen ohne neuen Erkenntnisstand.

2.2 • Die bestehende Datenbasis

Jeder Scan-Lauf schreibt vollständig in eine zentrale SQLite-Datenbank: Signalköpfe und Kennzahlen, unveränderliche Trade-Snapshots (SHA-256-adressiert, jederzeit diff-bar), Forensik-Befunde als JSON, sämtliche KI-Berichte mit Modell und Token-Verbrauch, Abonnenten-Verläufe, gespiegelte Testberichte des MqIDownloader-Dienstes sowie — seit Kurzem — eine append-only Ampel-Chronik mit protokollierten Farb- und Kriteriumswechseln. Stand September 2026 umfasst die Basis 61 Signale, 59 vollständige Forensiken, 674 KI-Analysen und 47 ausführliche Tiefenanalysen.

Auf dieser Basis ruht ein zweistufiges LLM-Verfahren: ein schnelles Modell für die Massen-Profile im Scan, ein starkes Modell für die Finalisten-Berichte. Die Sprachmodelle erhalten ausschließlich fertig gerechnete Befund-JSONs — keine Roh-Trades, keine Zugangsdaten — und der Engine-Befund ist für ihre Texte bindend.

2.3 • Der Restaufwand des Menschen

Was bleibt, ist Bedienung: Läufe starten, Ergebnisse sichten, bei Auffälligkeiten in Detailansichten abtauchen, Berichte lesen, Abonnenten-Verläufe interpretieren, den Markt im Blick behalten. Jede dieser Tätigkeiten folgt einer wiederkehrenden Logik — und genau solche Logik lässt sich an einen Agentenbetrieb übertragen.

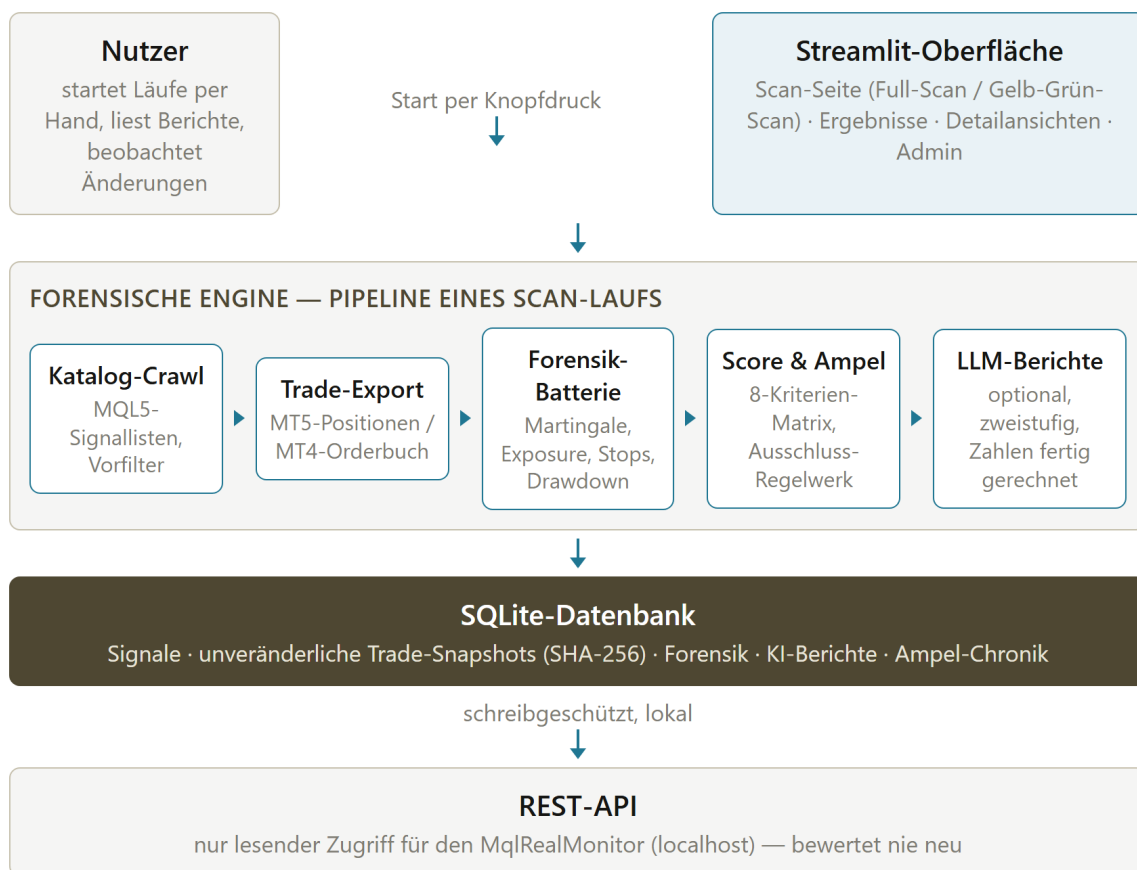


Abbildung 1: Ist-Architektur — Engine-Pipeline und Datenfluss eines manuell gestarteten Scan-Laufs

2.4 · Prinzipien als Fundament

Vier Erfahrungen der Analyse-Reihe sind für das Folgende tragend, weil sie die Fehlerquellen von Sprachmodellen im Finanzkontext direkt adressieren:

Code rechnet, das LLM interpretiert. Arithmetik in Sprachmodellen halluziniert; alle Kennzahlen entstehen daher deterministisch in der Engine und werden im Prompt nur zitiert.

Die Engine ist bindend. Ein Bericht darf eine Ampel nie aufwerten — ausgeschlossene oder rote Signale bleiben abgelehnt, egal wie schön die Einzeldaten aussehen.

Append-only statt Überschreiben. Chroniken, Snapshots und Protokolle werden nur ergänzt, nie nachträglich verändert — Nachvollziehbarkeit ist Teil des Produkts.

Sichtbarkeit statt Blackbox. Jede Hintergrundaktivität ist in der Oberfläche sichtbar; der Nutzer entscheidet, niemals der Agent.

3 · Die Idee: Beobachtung als Dienst

Der Agentenbetrieb ersetzt nicht die Engine — er setzt auf sie. An die Stelle des manuellen Anstoßens tritt ein Dienst, der nach festem Takt arbeitet; an die Stelle des Lesens vieler Einzelseiten tritt eine Bewertung, die nur noch bei Abweichungen laut wird. Die Gegenüberstellung zeigt, was sich ändert:

Tätigkeit	Heute (manuell)	Künftig (Agentenbetrieb)
Scans starten	Full- und Überwachungsläufe per Knopfdruck	Dirigent stößt Läufe nach Wochen- und Monatsraster an
Markt beobachten	nebenbei, intuitiv	Marktbeobachter liefert täglichen Kennzahlen-Kontext aus dem MetaTrader
Neue Trades prüfen	Detailansicht öffnen, Vergleiche anstellen	Betreuer prüft das Trade-Delta gegen das Algo-Profil
Berichte lesen	viele Seiten pro Lauf	Lagebericht fasst die Woche; Alerts nennen nur Abweichungen
Änderungen bemerken	beim zufälligen Blick	Ampel-Wechsel-Protokoll und Stilbruch-Flags lösen sofortige Meldung aus

Tabelle 2: Arbeitsanteile vor und nach der Übergabe an den Agentenbetrieb

Der Schlüssel zur Automatisierung liegt im Wissen über die Strategien. Die vorhandenen Tiefenanalysen beschreiben je Signal den Handelsalgorithmus in der Tiefe: Ausbruchs-Scanner mit News-Fenstern, Grid-Körbe, Rollover-Scalper, diskretionäre manuelle Trader. Diese Beschreibungen werden zu kompakten, **prüfbar**en Profilen verdichtet — mit expliziten Konformitäts-Merkmalen („handelt überwiegend im Fenster um 21:10 Uhr“, „Lot konstant bei 0,01“) und Warn-Merkmalen („nachts Positionen wäre eine Abweichung“, „steigende Lots nach Verlusten“). Damit wird aus einem Textbericht eine Messvorschrift für den täglichen Abgleich.

Abgrenzung. Der Agentenbetrieb trifft keine Anlage-Entscheidungen und bewertet nichts neu. Er verdichtet Beobachtung. Die Entscheidung, ein Signal zu abonnieren oder zu kündigen, bleibt beim Menschen — unterstützt durch Lageberichte und Alerts.

4 · Architektur des Agentenbetriebs

Der Agentenbetrieb wird als **eigener Dienstprozess im selben Projekt** realisiert: ein Daemon, der dieselben Engine-Module wie die graphische Oberfläche als Bibliothek nutzt und in dieselbe Datenbank schreibt. Die Prozesse sind getrennt — die Oberfläche kann geschlossen werden, der Betrieb läuft weiter — aber Projekt, Konfiguration, Secrets und Datenbestand sind gemeinsam.

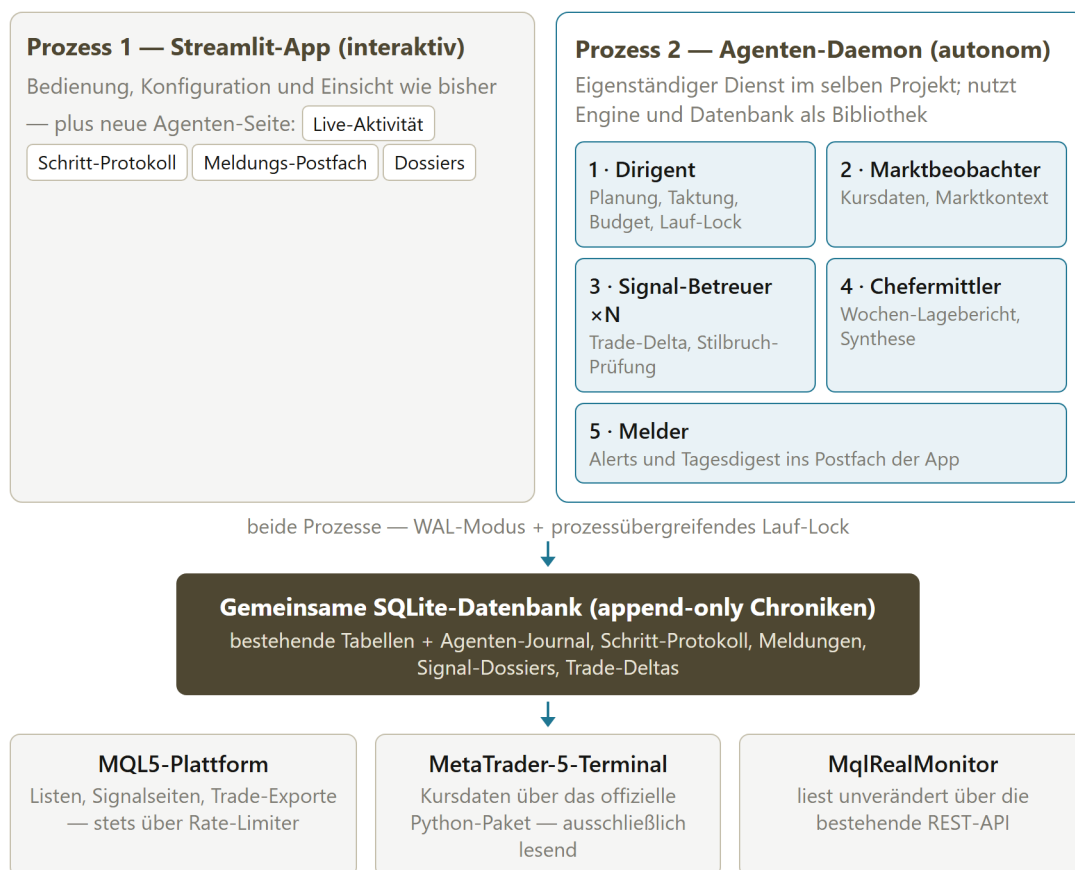


Abbildung 2: Ziel-Architektur — zwei Prozesse, eine Datenbank; der Daemon nutzt Engine und Forensik als Bibliothek

Drei technische Entscheidungen tragen die Konstruktion:

SQLite mit WAL und Lauf-Lock. Zwei Prozesse teilen sich eine Datenbankdatei; Write-Ahead-Logging und kurze Transaktionen machen das stabil, ein prozessübergreifendes Lauf-Lock verhindert, dass Agenten-Scan und manueller Oberflächen-Scan sich gleichzeitig bei MQL5 und in der Datenbank in die Quere kommen.

Keine neue Schreib-API. Die bestehende REST-Schnittstelle bleibt ausschließlich lesend und lokal. Ein separates Steuerprogramm über HTTP wäre technisch möglich, verlangte aber authentifizierte Schreib-Endpunkte — Angriffsfläche und Doppelpflege ohne Nutzen in diesem Szenario.

Derselbe Code-Pfad für Bewertung. Was der Dirigent anstößt, ist der identische Pipeline-Lauf, den heute die Oberfläche startet — inklusive Rate-Limiting, Fehlerbehandlung und Ampel-Chronik. Der Agentenbetrieb erhält keinen zweiten, abweichenden Bewertungsweg.

5 · Die fünf Rollen

Fünf spezialisierte Rollen teilen sich die Arbeit. Jede Rolle hat ihr eigenes Prompt-Template (editierbar wie die bestehenden Berichts-Prompts), ihren Takt und ihre Zuständigkeit; als Standardmodell ist je Rolle **GLM-5.3** vorkonfiguriert, pro Rolle umstellbar auch auf das kleinere Flash-Modell oder beliebige OpenAI-kompatible Endpunkte.

Rolle	Takt	Auftrag	Ergebnis
1 · Dirigent	täglich, werktags	Betrieb planen: Budget, Sperren, Markt-Kalender; Läufe anstoßen	Tagesplan, angestoßene Scans, Budget-Überwachung
2 · Marktbeobachter	täglich	Kursdaten aus dem MetaTrader holen; Code berechnet Kennzahlen	Marktkontext (JSON und Kurzfassung) je Symbol
3 · Signal-Betreuer (einer je Kandidat)	täglich	Trade-Delta holen und gegen das Algo-Profil prüfen	Beobachtung: konform, auffällig oder Stilbruch
4 · Chefermittler	wöchentlich, monatlich	Synthese über alle Dossiers, Marktkontext und Wechsel-Protokoll	Lagebericht mit Empfehlungen (keine Neubewertung)
5 · Melder	bei Ereignis und täglich	Journal verdichten, Eskalationen formulieren	Alerts und Tagesdigest im Postfach der App

Tabelle 3: Rollen, Takt und Auftrag

Zwei Rollen verdienen Hervorhebung. Der **Dirigent** ist überwiegend deterministischer Code — der Ablaufplan steht fest, das Sprachmodell entscheidet nur Randfragen. Seine Entscheidungen sind auf eine Whitelist erlaubter Aktionen begrenzt und werden vor Ausführung vom Code validiert; damit kann kein Modellaufruf je einen unvorhergesehenen Eingriff veranlassen. Der **Signal-Betreuer** ist die eigentliche Arbeitsstelle: Er hält das Dossier seines Signals aktuell, prüft jeden Zuwachs an Trades und eskaliert Abweichungen. Bei einem Stilbruch-Verdacht wiederholt er die Prüfung mit erhöhtem Aufwand, bevor der Melder alarmiert.

Die Rollen kommunizieren nicht in freiem Chat, sondern über ein **append-only Agenten-Journal** in der gemeinsamen Datenbank: jede Rolle schreibt strukturierte Einträge, jede liest die für sie bestimmten. Das ist auditierbar, token-sparend und passt zur Philosophie des Projekts — der Nutzer kann jede Zeile der Verständigung nachlesen.

6 · Das Signal-Dossier

Jedes beobachtete Signal erhält eine ständig wachsende Datenbasis — das Dossier. Es bündelt vier Schichten, die aus unterschiedlichen Quellen gespeist werden und gemeinsam die Frage der Tagesprüfung beantworten:

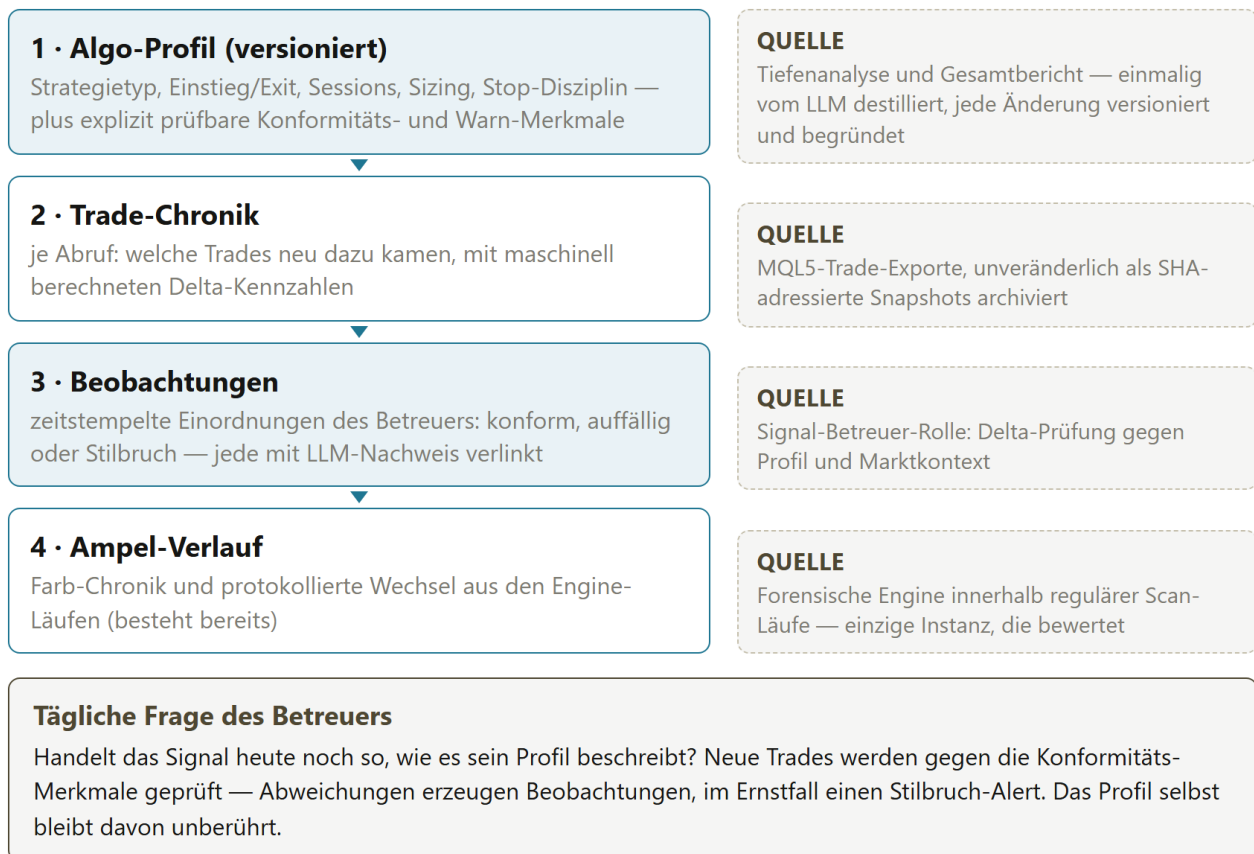


Abbildung 3: Aufbau des Signal-Dossiers — vier Schichten mit Quellen

Das **Algo-Profil** im Kopf des Dossiers entsteht einmalig durch Destillation aus Tiefenanalyse und Gesamtbericht und wird nur mit begründeter neuer Version geändert — etwa nach einer aufgefrischten Tiefenanalyse oder einem bestätigten Strategiewechsel des Anbieters. Die **Trade-Chronik** darunter hält je Abruf fest, welche Trades neu hinzukamen, samt der dazu berechneten Delta-Kennzahlen. Die **Beobachtungen** sind die zeitstempelten Urteile des Betreuers, jede mit Verweis auf den protokollierten Modellaufruf. Den Boden bildet der bereits existierende **Ampel-Verlauf** aus den Engine-Läufen.

Für die 47 vorhandenen Tiefenanalysen lassen sich die Profile rückwirkend erzeugen — der Agentenbetrieb startet also nicht bei null, sondern mit einer fertigen Wissensbasis über alle Kandidaten der Gesamtanalyse.

7 • Der Tagesablauf: Trade-Delta-Prüfung

Das Herzstück des Betriebstags ist die Prüfung neuer Trades. Weil jeder Trade-Export als unveränderlicher, SHA-256-adressierter Snapshot archiviert ist, ist die Differenz zum Vortag exakt und billig feststellbar — ein Abgleich zweier Hashwerte genügt, um zu wissen, ob überhaupt etwas passiert ist:

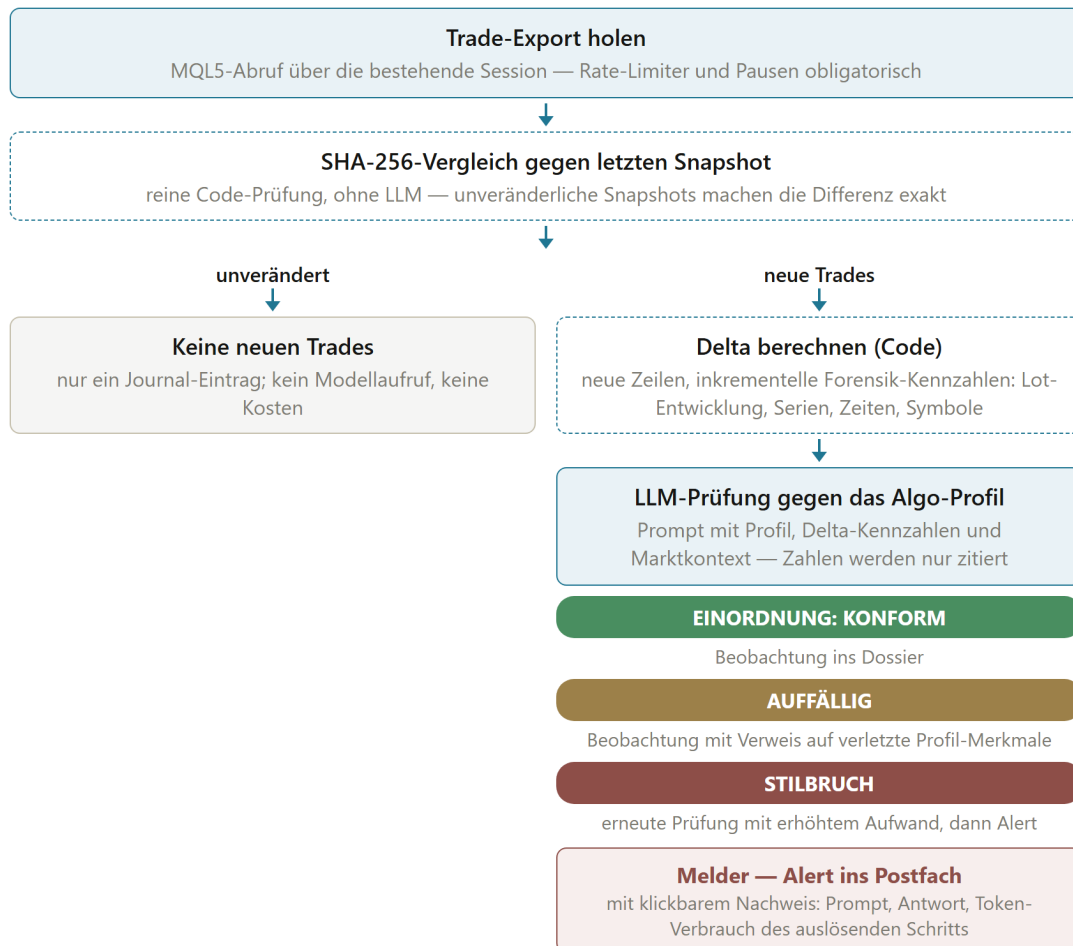


Abbildung 4: Trade-Delta-Prüfung — nur bei neuen Trades läuft ein Modellaufruf; Einordnungen sind Beobachtungen, keine Neubewertungen

Ein **Stilbruch** liegt vor, wenn das Verhalten systematisch vom Profil abweicht: nächtliche Positionen bei einem Ausbruchs-Scanner mit festen Handelsfenstern, wachsende Lots nach Verlustserien bei sonst konstantem Sizing, neue Symbole oder ein Verschwinden der bisher nachweisbaren Stop-Disziplin. Einzelne Ausreißer sind noch kein Stilbruch — die Einordnung „auffällig“ dokumentiert sie zur Beobachtung; erst das Muster eskaliert. Die Schwelle zwischen Beobachtung und Alarm steht im Profil selbst, als nummerierte Warn-Merkmale.

8 · Marktdaten aus dem MetaTrader

Für die Marktbeobachtung greift der Agentenbetrieb auf das Terminal des Nutzers zu — über das offizielle Python-Paket der MetaTrader-5-Plattform. Es verbindet sich direkt mit einem laufenden Terminal und stellt Kurs- und Bar-Daten bereit (Zeitreihen je Symbol und Timeframe, aktuelle Kurse), ohne dass irgendetwas im Terminal installiert oder verändert werden müsste.

Die Anbindung ist bewusst minimal gehalten: **ausschließlich Lese-Funktionen** sind angebunden — Verbindung aufbauen, Symbol aktivieren, Kursreihen lesen, Verbindung trennen. Order-Funktionen sind nicht verdrahtet und werden von einem statischen Test bewacht. Das Terminal selbst startet der Daemon standardmäßig **nicht**: Läuft es nicht, wartet der Marktbeobachter bis zum nächsten Intervall und der Melder vermerkt die Lücke im Digest. Ob der Daemon ein ausgeschaltetes Terminal selbst starten darf, ist ein

Konfigurationsschalter — Voreinstellung ist nein, der Pfad zum Terminal ist austauschbar.

Aus den Rohkursen berechnet Code die eigentlichen Kennzahlen — Bewegung über Tag, Woche und Monat, Volatilität, Trendlage, Distanz zu Hoch und Tief, relevante Zeitfenster. Das Sprachmodell erhält diese fertigen Zahlen und liefert die Kurzfassung der Marktlage, die dann in die Betreuer-Prüfungen einfließt: Handelt ein Ausbruchs-Scanner in einer ruhigen Phase auffällig viel, ist das eine andere Information als derselbe Handel nach einem Volatilitätsschock.

9 • Betriebsrhythmus und Kosten

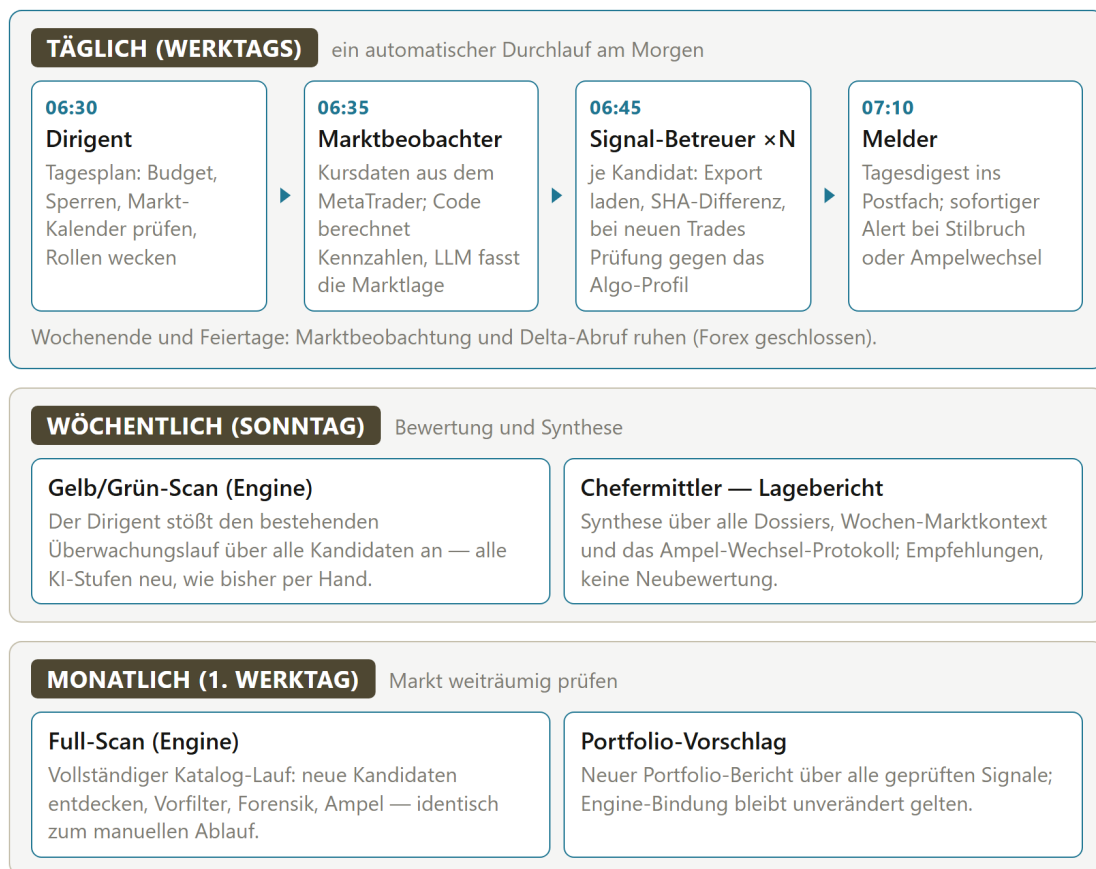


Abbildung 5: Betriebsrhythmus über Tag, Woche und Monat — der Dirigent stößt ausschließlich bestehende Lauf-Typen an

Die Taktung folgt der Natur des Marktes: Werktag für Werktag die Delta-Prüfung, sonntags die Neubewertung durch den Überwachungslauf über alle Kandidaten plus Lagebericht, monatlich der vollständige Katalog-Scan mit Portfolio-Vorschlag. Wochenenden und Feiertage ruhen. Jede Aktivität respektiert das Lauf-Lock gegenüber der Oberfläche und die Rate-Limits gegenüber MQL5.

Die Kosten bleiben beherrscht, weil Modellaufrufe an Bedingungen geknüpft sind: kein Aufruf ohne neue Trades, kein Starkmodell ohne Anlass, und sämtliche Berechnung läuft ohne Tokens. Zwei Budgets deckeln den Betrieb — ein Tagesbudget und ein Monatsbudget; bei Erschöpfung werden Modell-Schritte übersprungen und gemeldet, Code-Schritte laufen weiter. Die regulären Scan-Läufe zählen auf ihr bestehendes eigenes Lauf-Budget.

500 Tsd.Token-Tagesbudget
aller Rollen zusammen**5 Mio.**Token-Monatsbudget
des Agentenbetriebs**0**Token, wenn keine
neuen Trades vorliegen

10 · Governance: Leitplanken autonomer KI

Autonomie ohne Kontrolle wäre hier fehl am Platz — die Rollen arbeiten deshalb unter vier eisernen Regeln, die sich aus den Prinzipien des Projekts ableiten:

- 1 — Keine Neubewertung.** Ampeln, Urteile und Scores entstehen ausschließlich in Engine-Läufen. Agenten beobachten, ordnen ein und melden; ihr Vokabular kennt keine Bewertungsbegriffe jenseits von Beobachtungskategorien.
- 2 — Zahlen nur aus dem JSON.** Jedes Modell erhält fertig gerechnete Kennzahlen mit Quellenangabe. Eigene Rechnungen im Prompt sind untersagt; Widersprüche zwischen Text und Daten gelten zugunsten der Daten und werden ausgewiesen.
- 3 — Whitelist statt Freiheit.** Der Dirigent kann nur Aktionen aus einer festen, im Code validierten Menge anstoßen. Modell-Entscheidungen sind Vorschläge innerhalb dieses Rahmens, niemals Eingriffe darüber hinaus.
- 4 — Lückenloses Protokoll.** Jeder Schritt — ob Code oder Modell — wird append-only festgehalten; bei Modellaufrufen mit vollständigem gefülltem Prompt, vollständiger Antwort, Modell, Token-Verbrauch und Dauer. Es gibt bewusst keinen „Trockenmodus“: die Transparenz des Protokolls ersetzt ihn.

Alles, was ein Modell sagt, steht im Protokoll. Die Agenten-Seite der Oberfläche zeigt den Betrieb live, jedes Gespräch aufklappbar bis hinunter zum gefüllten Prompt. Meldungen verweisen klickbar auf die auslösenden Schritte.

11 · Datenmodell: sechs neue Tabellen

Der Agentenbetrieb ergänzt die bestehende Datenbank um sechs Tabellen — alle append-only, keine überschreibt bestehende Daten:

Tabelle	Inhalt
agenten_laeufe	je Zyklus einer Rolle: Zeitraum, Status, Zusammenfassung
agenten_schritte	je Code- oder Modell-Schritt: Art, Prompt und Antwort im Volltext, Modell, Token, Dauer — das Protokoll
agenten_meldungen	Alerts, Digests und Lageberichte mit Quellenverweisen
dossier_profil	Algo-Profil je Signal, versioniert mit Begründung
dossier_beobachtungen	Einordnungen des Betreuers je Signal, verlinkt auf Delta und Modell-Nachweis
trade_deltas	je Abruf: alter und neuer Snapshot-Hash, Anzahl neuer Trades, Delta-Kennzahlen

Tabelle 4: Neue Tabellen des Agentenbetriebs (ergänzend zu den acht bestehenden)

12 · Benutzeroberfläche

Die Bedienung bleibt grafisch und einfach — drei Neuerungen in der gewohnten Streamlit-Oberfläche:

Admin-Bereich „Agenten und LLMs“. Fünf Rollenkarten mit Schalter, Modell-Auswahl (GLM-5.3 vorkonfiguriert), Token-Limit und Takt; dazu die Prompt-Vorlagen je Rolle mit Editor, Zurücksetzen-Möglichkeit und Änderungsmarkierung — bekannt vom bestehenden Prompt-Bereich. Globalschalter starten und stoppen den Betrieb.

Admin-Bereich „Marktdaten“. Terminal-Pfad (austauschbar), Start-Politik, Symbol-Beobachtungsliste (automatisch aus den Kandidaten abgeleitet, manuell erweiterbar), Timeframes und Abrufzeit — plus ein Verbindungstest, der eine Kursreihe liest und das Ergebnis anzeigt.

Seite „Agenten“. Live-Ansicht der laufenden Rollen, das Schritt-Protokoll mit Filtern nach Rolle, Signal und Tag, das Meldungs-Postfach mit Alerts und Lageberichten sowie die Dossier-Ansicht je Signal. Diese Seite ist das Fenster zu den Sprachmodellen: Jeder aufruf ist bis zum vollständigen Wortlaut nachlesbar.

13 · Umsetzungsfahrplan

Fünf Stufen, jede mit hartem Abnahmekriterium — erst wenn die Kriterien stehen, ist die nächste Stufe dran:

Stufe	Inhalt	Abnahmekriterium
A — Fundament	Daemon, Scheduler, Lauf-Lock, Journal-Tabellen, Admin-Bereich, Agenten-Seite mit Protokoll	ein Dirigent-Tageslauf vollständig im Protokoll nachlesbar
B — Dossiers	Profil-Destillation (rückwirkend aus den Tiefenanalysen), Trade-Delta, Betreuer-Rolle mit Stilbruch-Flag	jeder Kandidat hat ein Dossier; ein Tag mit neuen Trades erzeugt je Signal eine Beobachtung
C — Marktdaten	MetaTrader-Anbindung (nur lesend), Marktkontext, Einbindung in Betreuer- und Chef-Prompts	tägliche Marktlage im Protokoll; Beobachtungen zitieren den Kontext
D — Melder	Postfach, Alerts auf Ampel-Wechsel, Stilbruch und Fehler; Tagesdigest	ein ausgelöster Wechsel erzeugt einen Alert mit Quellverweis
E — Chefermittler	Wochen-Lageberichte; Dirigent stößt Überwachungs- und Vollscans selbst an	ein kompletter Monat ohne einen manuellen Scan-Klick

Tabelle 5: Ausbaustufen mit Abnahmekriterien

Stufe A erledigt nebenbei das älteste offene Betriebspunkt des Projekts — den Wiederholungslauf von der Kommandozeile aus. Jede Stufe schließt mit Tests (Unit- und Oberflächen-Tests), Dokumentations-Nachzug und Versionierung ab.

14 · Risiken und Gegenmaßnahmen

Risiko	Gegenmaßnahme
Token-Kosten des Dauerbetriebs	Tages- und Monatsbudget; kein Aufruf ohne neue Trades; kleinere Modelle je Rolle wählbar
Accountsperre bei übermäßigem Abruf (MQL5-Nutzungsbedingungen)	durchgängig bestehender Rate-Limiter mit Pausen und Jitter; Exportfenster gestreut; Fail-Fast bei Drosselung
Schreibkonflikte in der gemeinsamen Datenbank	WAL-Modus, kurze Transaktionen, prozessübergreifendes Lauf-Lock
Verbindung zum Terminal scheitert (Rechte, Bitbreite, mehrere Installationen)	Verbindungstest mit klarer Diagnose; Warteschleife und Digest-Hinweis statt Stillstand; Terminal-Pfad austauschbar
Unbeabsichtigter Terminal-Start	Start-Politik standardmäßig aus; Live-Terminal wird nie ohne Konfiguration gestartet
Fehlleistungen der Sprachmodelle	Zitierpflicht aus JSON, Whitelist-Aktionen, Engine-Bindung, vollständiges Protokoll zur nachträglichen Prüfung

Tabelle 6: Wesentliche Risiken und ihre Begrenzung

15 · Fazit und Ausblick

Der MqlKiScanner hat bewiesen, dass sich MQL5-Signale forensisch prüfen lassen — reproduzierbar, auf Rohdaten, mit bindenden Regeln. Der Agentenbetrieb überträgt dieses Vertrauen auf die Dauerbeobachtung: Fünf Rollen mit klaren Aufträgen, ein Journal statt Chat, Profile statt Vermutungen und ein Protokoll, das jede Äußerung der Modelle festhält. Der Mensch behält zwei Entscheidungen: die Konfiguration des Betriebs und die Anlage-Entscheidung selbst.

Mittelfristig liegen die interessanteren Fragen jenseits dieses Papiers: der Abgleich des Kopier-Ergebnisses im eigenen Konto mit den Signal-Erwartungen (Abweichungs-Alerte), die Verlängerung der Profile um Vorhersagen über das Verhalten in definierten Marktszenarien und die Kommunikation mit den weiteren Werkzeugen des Ökosystems — Monitor, Downloader und Backtester — auf dem gleichen sauberen Weg wie hier beschrieben: lesend, protokolliert, ohne Neubewertung.

Projekt und Quelltext: github.com/tnickel/MqlKiScanner — der detaillierte Bauplan (Phase für Phase, Tabellen-Schemata, Prompt-Rohfassungen) liegt dort in der Projektdokumentation.